Doc-Genie

Professional Python Function Docstring Generator with AST Analysis

PROJECT DESCRIPTION

Doc-Genie is a smart Python documentation tool that automatically creates docstrings for Python functions. It uses Abstract Syntax Tree (AST) analysis to understand the structure of Python code and generate clear and useful documentation.

Writing documentation manually can take a lot of time and developers may forget to write it. Doc-Genie solves this problem by automatically analyzing Python functions and generating professional docstrings. It supports different documentation styles such as Google style and NumPy style**.**

The system reads Python code, identifies functions, parameters, and return values, and then creates proper documentation for them. It helps developers understand the code easily and keeps documentation consistent.

Doc-Genie also provides a simple interface using **Gradio**, where users can paste their code and generate documentation quickly. The generated documentation can also be exported as TXT or PDF files**.**

Overall, Doc-Genie makes Python documentation easier, faster, and more organized for developers.

SCENARIOS

Scenario 1: Legacy Code Documentation

A developer receives an old Python project with many functions but no proper documentation. Using Doc-Genie, they upload the Python files and automatically generate docstrings for all functions. The tool analyzes the code using AST and identifies parameters, return values, and function behavior. This helps the developer quickly document the entire project.

Scenario 2: API Development

A backend developer creating APIs needs proper documentation for each function. Doc-Genie analyzes the functions and generates NumPy or Google style docstrings automatically. This ensures that API documentation is clear and consistent.

Scenario 3: Team Collaboration

A team of developers wants to maintain the same documentation style in all their projects. Doc-Genie helps them generate standardized docstrings. The documentation can also be exported as PDF reports and shared with team members for review.

Scenario 4: Educational Purpose

A teacher uses Doc-Genie to teach students how to write proper documentation. Students write Python functions and the tool generates professional docstrings. This helps them learn how good documentation should look.

Scenario 5: Code Review

During code review, QA engineers check whether functions are properly documented. Doc-Genie generates expected docstrings for each function. Reviewers can compare them with existing documentation to ensure quality.

ARCHITECTURE OVERVIEW

Doc-Genie is designed as a modular system that combines Python's AST module with **Gradio** to generate automatic documentation for Python code. The system focuses on accurate code analysis, high-quality documentation, and a simple user interface.

The tool reads Python functions, analyzes them using AST, and then generates proper docstrings. The Gradio interface allows users to easily input code and view the generated documentation.

CORE TECHNOLOGIES

- **Python 3.8+** – Main programming language used for development
- **AST (Abstract Syntax Tree)** – Used to analyze the structure of Python code
- **Gradio 4.0+** – Used to create an interactive web interface
- **ReportLab** – Used to generate documentation reports in PDF format
- **Regular Expressions** – Used for pattern matching in code analysis
- **Google Colab** – Used for development and running the project online
- **HTML5 / CSS3** – Used for basic interface styling

PRE-REQUISITES

Software Requirements

- **Python 3.8 or above**
- **Google Account** (for accessing Google Colab)
- **pip** (Python package manager)
- **Web Browser** such as Chrome, Firefox, Safari, or Edge
- **Internet connection** to run the project in Colab and access Gradio

KNOWLEDGE PREREQUISITES

To use Doc-Genie, users should have basic knowledge of the following:

- **Python Basics** – Functions and basic programming concepts
- **Python AST** – Basic idea of how Python code structure is analyzed
- **Python Docstrings** – Understanding of Google or NumPy style documentation
- **Gradio Framework** – Basic idea of creating simple interfaces
- **Code Analysis** – Understanding how code can be analyzed automatically
- **Google Colab** – Basic knowledge of running notebooks and executing cells

---

HARDWARE REQUIREMENTS

- **Processor:** Any modern processor (Intel i3 or similar)
- **RAM:** Minimum 2 GB RAM
- **Storage:** Around 500 MB free space
- **Display:** Minimum resolution of 1024 × 768 for proper interface display

PROJECT WORKFLOW

Phase 1: Environment Setup

Activity 1.1: Install Required Tools

- Install all required packages using requirements.txt
- Set up the project in Google Colab
- Install packages such as Gradio, Transformers, Torch, ReportLab
- Check if all packages are installed correctly

Activity 1.2: Create Main Analyzer Class

- Create a class called DocGenieAnalyzer
- Use Python AST to analyze Python functions
- Detect function parameters and return types
- Store generated documentation history

Phase 2: Code Analysis

Activity 2.1: Function Signature Extraction

- Extract the function name
- Identify parameters and their types
- Detect default values of parameters
- Identify the return type of the function

Activity 2.2: Function Logic Analysis

- Analyze the function body using AST
- Detect loops (for, while)
- Detect conditions (if, else)
- Identify mathematical operations
- Detect return statements
- Identify function calls
- Generate a simple description of the function

Phase 3: Interface Development

Activity 3.1: Create User Interface

- Create a code input box for Python code
- Allow users to upload Python (.py) files
- Add option to select Google or NumPy docstring style
- Display the generated documentation
- Add a Generate button to run the analysis

Activity 3.2: Export Features

- Export documentation as PDF
- Export documentation as TXT file
- Allow users to download the files
- Show export status messages

Phase 4: Reporting System

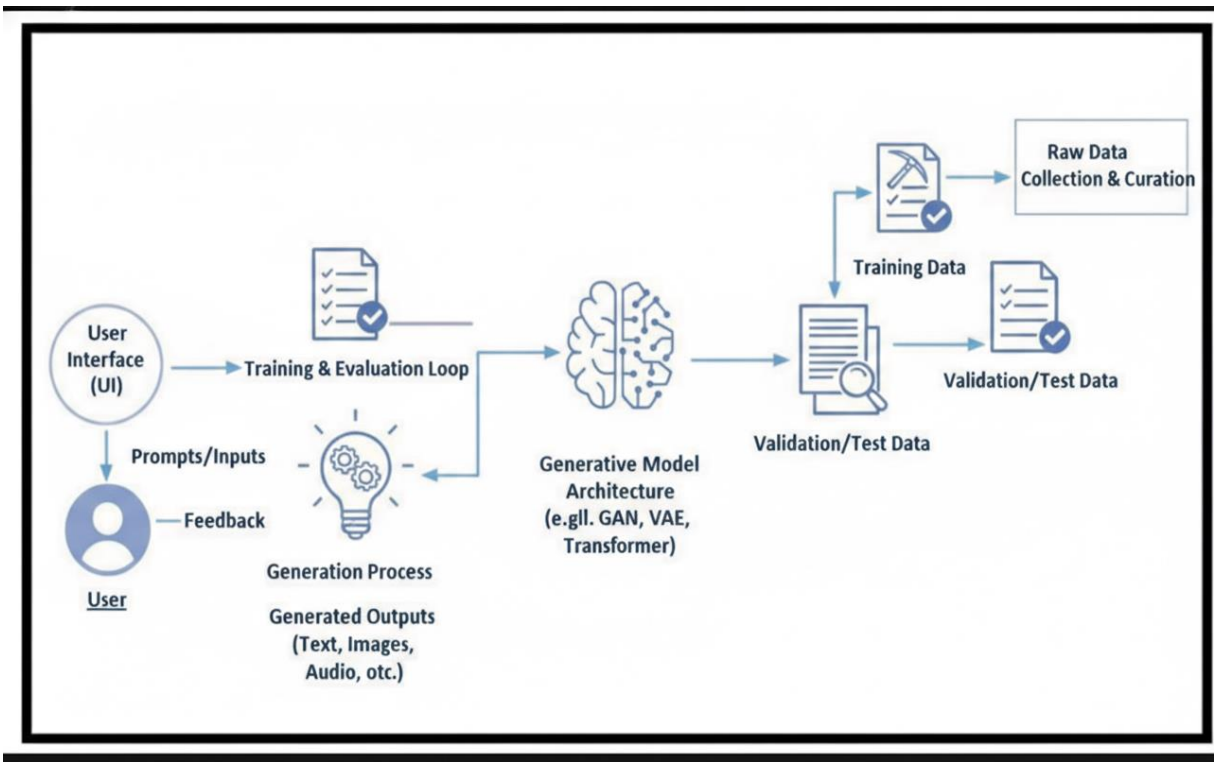
Activity 4.1: PDF Generation

- Use ReportLab to create PDF documents
- Format documentation clearly in the PDF
- Add page numbers and metadata

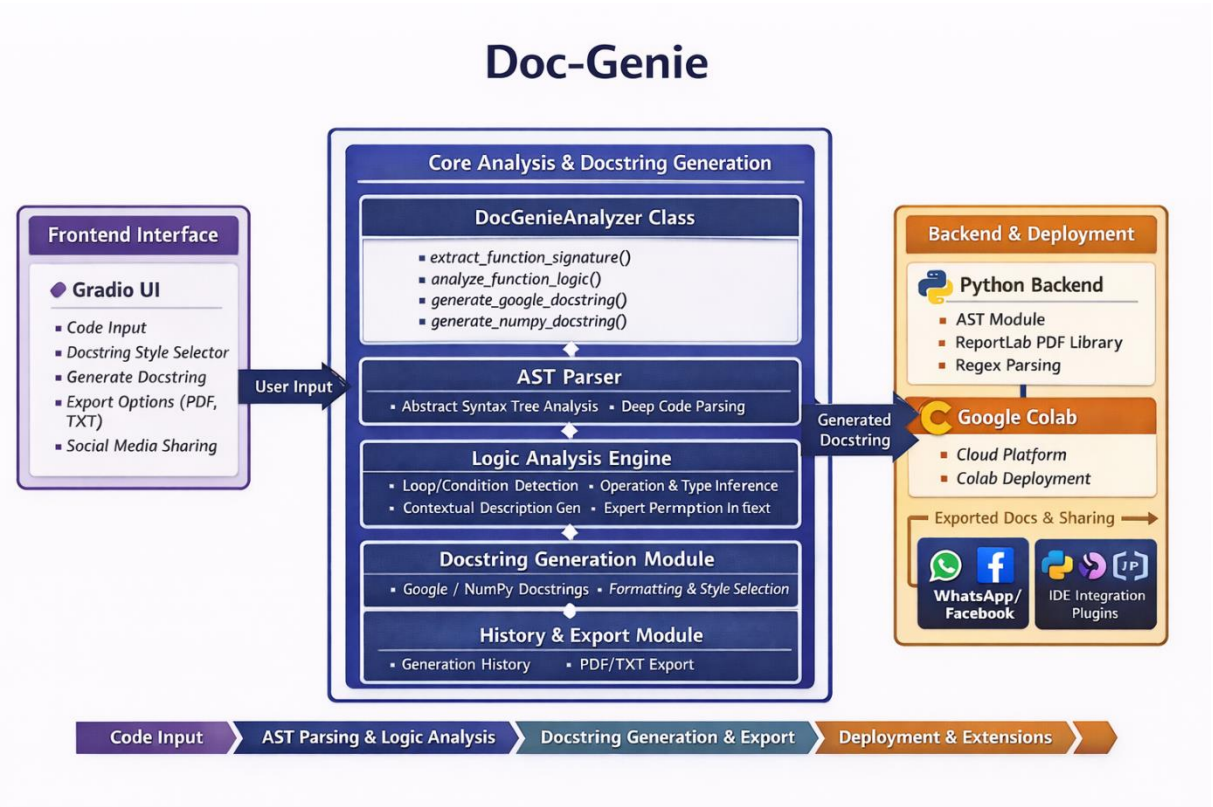
Activity 4.2: Text File Export

- Generate formatted text documentation
- Add clear sections and headings
- Include analysis details in the file

TECHNICAL ARCHITECTURE:



Project Structure



MILESTONE 1: Environment Setup & Core Architecture

Activity 1.1: Install Dependencies

Create requirements.txt with the following dependencies:

- gradio==4.41.0
- transformers==4.35.0
- torch==2.0.0
- reportlab==4.0.7
- requests==2.31.0
- black==24.1.1
- autopep8==2.1.0

Install via: pip install -r requirements.txt

```
[7]
3s
!pip install -q gradio transformers torch black autopep8 reportlab requests ast-super-parser

import gradio as gr
from datetime import datetime
from pathlib import Path
import json
import re
import ast
from reportlab.lib.pagesizes import letter, A4
from reportlab.platypus import SimpleDocTemplate, Paragraph, Spacer, PageBreak, Table, TableStyle, Preformatted
from reportlab.lib.styles import getSampleStyleSheet, ParagraphStyle
from reportlab.lib.units import inch
from reportlab.lib import colors
import requests
import urllib.parse
import inspect
```

Code reference (hint: click on image and you can copy paste from Comment)

Activity 1.2: Initialize Core Analyzer Class

Create DocGenieAnalyzer class with following structure:

class DocGenieAnalyzer:

@staticmethod
def extract_function_signature(code):
- Parse AST from code
- Extract function name, parameters, return type
- Return signature dictionary

@staticmethod
def analyze_function_logic(func_def, code):
- Walk through AST nodes
- Detect loops, conditions, operations
- Generate contextual description
- Return analysis dictionary

@staticmethod
def generate_google_docstring(signature, analysis, code):
- Format Google-style docstring
- Include summary, args, returns
- Return formatted string

@staticmethod
def generate_numpy_docstring(signature, analysis, code):
- Format NumPy-style docstring
- Include summary, parameters, returns
- Return formatted string

```python
class DocGenieAnalyzer:
    """Analyze Python functions and generate professional docstrings"""

    @staticmethod
    def extract_function_signature(code):
        """Extract function name, parameters, return type from code"""
        try:
            tree = ast.parse(code)
            func_def = tree.body[0]
            if not isinstance(func_def, ast.FunctionDef):
                return None, None

            func_name = func_def.name
            params = []

            for arg in func_def.args.args:
                param_type = "Any"
                if arg.annotation:
                    param_type = ast.unparse(arg.annotation) if hasattr(ast, 'unparse') e
                params.append({
                    'name': arg.arg,
                    'type': param_type,
                    'default': None
                })
```

Code reference (hint: click on image and you can copy paste from Comment)

```python
            return None, None

    @staticmethod
    def analyze_function_logic(func_def, code):
        """Deep analysis of function body to understand actual behavior"""
        try:
            body = func_def.body
            analysis = {
                'has_loops': False,
                'has_conditions': False,
                'has_return': False,
                'loop_types': [],
                'condition_types': [],
                'operations': [],
                'return_types': [],
                'variables_used': set(),
                'function_calls': [],
                'description': ""
            }

            # Walk through AST
            for node in ast.walk(func_def):
                if isinstance(node, ast.For):
                    analysis['has_loops'] = True
                    analysis['loop_types'].append('for')
```

Code reference (hint: click on image and you can copy paste from Comment)

```python
        elif isinstance(node, ast.While):
            analysis['has_loops'] = True
            analysis['loop_types'].append('while')
        elif isinstance(node, ast.If):
            analysis['has_conditions'] = True
            analysis['condition_types'].append('if')
        elif isinstance(node, ast.Return):
            analysis['has_return'] = True
            if node.value:
                analysis['return_types'].append(ast.unparse(node.value) if hasattr(ast, 'unparse') else
        elif isinstance(node, ast.BinOp):
            op = node.op
            if isinstance(op, ast.Mult):
                analysis['operations'].append('multiplication')
            elif isinstance(op, ast.Add):
                analysis['operations'].append('addition')
            elif isinstance(op, ast.Sub):
                analysis['operations'].append('subtraction')
            elif isinstance(op, ast.Div):
                analysis['operations'].append('division')
        elif isinstance(node, ast.Call):
            if isinstance(node.func, ast.Name):
                analysis['function_calls'].append(node.func.id)
        elif isinstance(node, ast.Name):
            analysis['variables_used'].add(node.id)

    # Build detailed description
    description_parts = []

    if func_def.name:
        description_parts.append(f"Executes {func_def.name} operation")

    if analysis['has_conditions']:
        description_parts.append("with conditional logic")

    if analysis['has_loops']:
        description_parts.append("and iterates through values")

    if 'multiplication' in analysis['operations'] and 'subtraction' in analysis['operations']:
        description_parts.append("performing mathematical calculations")
    elif 'multiplication' in analysis['operations']:
        description_parts.append("using multiplication operations")
    elif 'addition' in analysis['operations']:
        description_parts.append("using addition operations")
    elif 'division' in analysis['operations']:
        description_parts.append("using division operations")

    if analysis['return_types']:
        description_parts.append(f"and returns the computed result")

    analysis['description'] = " ".join(description_parts) + "."
    if len(analysis['description']) < 20:
        analysis['description'] = "Performs computation based on input parameters."

    return analysis
except Exception as e:
    return {
        'has_loops': False,
        'has_conditions': False,
        'has_return': False,
        'description': "Executes function logic based on provided parameters."
    }
```

Code reference (hint: click on image and you can copy paste from Comment)

MILESTONE 2: Advanced AST Analysis Engine

Activity 2.1: Function Signature Extraction

Extract comprehensive function information:

Input: def calculate(x: int, y: float = 3.5) -> float:

Extracted Information:

```
{
        'name': 'calculate',
        'params': [
        {'name': 'x', 'type': 'int', 'default': None},
        {'name': 'y', 'type': 'float', 'default': '3.5'}
        ],
        'return_type': 'float'
}
```

Code reference (hint: click on image and you can copy paste from Comment)

Activity 2.2: Deep Logic Analysis

Analyze function behavior through AST walking:

Detection Capabilities:
1. Loop Detection: Identifies for/while loops
2. Condition Detection: Finds if/elif/else statements
3. Operation Detection: Recognizes mathematical operations
4. Return Analysis: Tracks return statements and types
5. Function Calls: Identifies internal and external calls
6. Variables: Tracks variables used in function

Analysis Output:

```
{
        'has_loops': True,
        'has_conditions': True,
        'operations': ['multiplication', 'addition'],
        'function_calls': ['helper_func'],
        'description': 'Executes calculation with conditional logic...'
}
```

Code reference (hint: click on image and you can copy paste from Comment)

MILESTONE 3: Docstring Generation & Formatting

Activity 3.1: Google-Style Docstring Generation

Generated Google-style docstring example:

```
"""Calculate factorial of a number.

Executes calculation with conditional logic and returns
the computed result.

Args:
n (int): Parameter for controlling n behavior.

Returns:
int: The result of the computation.
"""
```

Format includes:
- Single-line summary
- Blank line separation
- Args section with types and descriptions
- Returns section with type and description

```python
@staticmethod
def generate_google_docstring(signature, analysis, code):
    """Generate accurate Google-style docstring"""
    summary = analysis.get('description', 'Processes input and returns result.')

    docstring = f'    """{summary}\n'

    if signature['params']:
        docstring += '\n    Args:\n'
        for param in signature['params']:
            default_str = f" (default: {param['default']})" if param['default'] else ""
            docstring += f"        {param['name']} ({param['type']}): Parameter for controlling {param['name']} behavior{default_str}.\n"

    docstring += f"\n    Returns:\n        {signature['return_type']}: The result of the computation.\n"
    docstring += '    """\n'

    return docstring
```

Code reference (hint: click on image and you can copy paste from Comment)

Activity 3.2: NumPy-Style Docstring Generation

Generated NumPy-style docstring example:

"""Calculate factorial of a number.

Executes calculation with conditional logic and returns
the computed result.

Parameters
----
n : int
Parameter for controlling n behavior.

Returns
----
int
The result of the computation.

```
    @staticmethod
    def generate_numpy_docstring(signature, analysis, code):
        """Generate accurate NumPy-style docstring"""
        summary = analysis.get('description', 'Processes input and returns result.')

        docstring = f'    """{summary}\n'

        if signature['params']:
            docstring += '\n    Parameters\n    ----------\n'
            for param in signature['params']:
                default_str = f", default: {param['default']}" if param['default'] else ""
                docstring += f"    {param['name']} : {param['type']}{default_str}\n"
                docstring += f"        Parameter for controlling {param['name']} behavior.\n"

        docstring += f"\n    Returns\n    -------\n    {signature['return_type']}\n"
        docstring += f"        The result of the computation.\n"
        docstring += '    """\n'

        return docstring

analyzer = DocGenieAnalyzer()
generation_history = []
```

Code reference (hint: click on image and you can copy paste from Comment)

```python
def generate_docstring(code, style="google"):
    """Generate docstring for Python function with accurate analysis"""

    if not code.strip():
        return None, "❌ Empty code provided!"

    try:
        signature, func_def = analyzer.extract_function_signature(code)
        if not signature:
            return None, "❌ Invalid Python function code!"

        # Deep analysis of the function
        analysis = analyzer.analyze_function_logic(func_def, code)

        if style.lower() == "google":
            docstring = analyzer.generate_google_docstring(signature, analysis, code)
        else:
            docstring = analyzer.generate_numpy_docstring(signature, analysis, code)

        # Extract function definition line
        code_lines = code.split('\n')
        func_line = code_lines[0]

        # Build final function with docstring
        final_code = func_line + '\n' + docstring
        for line in code_lines[1:]:
            final_code += line + '\n'

        generation_history.append({
            'code': code,
            'docstring': docstring,
            'style': style,
            'timestamp': datetime.now().isoformat(),
            'analysis': analysis
        })

        return final_code, f"✅ {style.upper()} Docstring Generated!"

    except Exception as e:
        return None, f"❌ Error: {str(e)}"

def process_upload(file):
    """Process uploaded Python file"""
    if not file:
        return None, "❌ No file uploaded!"

    try:
        content = file.read().decode('utf-8')
        return content, "✅ File loaded successfully!"
    except:
        return None, "❌ Failed to read file!"

def clear_history():
    """Clear generation history"""
    global generation_history
    generation_history = []
    return "✅ History cleared!"

def download_pdf():
    """Download all generated docstrings as PDF"""
    if not generation_history:
        return "❌ No documentation generated!", None

    timestamp = datetime.now().strftime("%Y-%m-%d_%H-%M-%S")
    filename = f"doc_genie_{timestamp}.pdf"
```

Code reference (hint: click on image and you can copy paste from Comment)

```python
try:
    doc = SimpleDocTemplate(filename, pagesize=A4, topMargin=0.5*inch, bottomMargin=0.5*inch)
    styles = getSampleStyleSheet()
    story = []

    title_style = ParagraphStyle(
        'Title',
        parent=styles['Heading1'],
        fontSize=24,
        textColor='#FF6B6B',
        spaceAfter=20,
        alignment=1
    )

    code_style = ParagraphStyle(
        'Code',
        parent=styles['Normal'],
        fontSize=9,
        fontName='Courier',
        backColor='#F0F0F0',
        borderColor='#CCCCCC',
        borderWidth=1,
        borderPadding=6
    )

    story.append(Paragraph("📚 Doc-Genie Documentation Report", title_style))
    story.append(Paragraph(f"Generated: {datetime.now().strftime('%Y-%m-%d %H:%M:%S')}", styles['Norma
    story.append(Spacer(1, 0.3*inch))

    for i, gen in enumerate(generation_history, 1):
        story.append(Paragraph(f"<b>Documentation #{i}</b>", styles['Heading2']))
        story.append(Paragraph(f"<b>Style:</b> {gen['style'].upper()}", styles['Normal']))
        story.append(Spacer(1, 0.1*inch))

        story.append(Paragraph("<b>Original Code:</b>", styles['Normal']))
        story.append(Preformatted(gen['code'], code_style))
        story.append(Spacer(1, 0.15*inch))

        story.append(Paragraph("<b>Generated Docstring:</b>", styles['Normal']))
        story.append(Preformatted(gen['docstring'], code_style))
        story.append(Spacer(1, 0.3*inch))

        if i % 2 == 0 and i != len(generation_history):
            story.append(PageBreak())

    story.append(Spacer(1, 0.3*inch))
    story.append(Paragraph(f"Total Documentations: {len(generation_history)}", styles['Normal']))

    doc.build(story)
    return f"✅ PDF Downloaded: {filename}", filename
except Exception as e:
    return f"❌ Error: {str(e)}", None
```

Code reference (hint: click on image and you can copy paste from Comment)

```python
def download_txt():
    """Download all generated docstrings as TXT"""
    if not generation_history:
        return "❌ No documentation generated!", None

    content = f"Doc-Genie Documentation Report\nGenerated: {datetime.now()}\n{'='*80}\n\n"

    for i, gen in enumerate(generation_history, 1):
        content += f"Documentation #{i}\n"
        content += f"Style: {gen['style'].upper()}\n"
        content += f"Generated: {gen['timestamp']}\n"
        content += f"{'-'*80}\n\n"
        content += f"ORIGINAL CODE:\n{gen['code']}\n\n"
        content += f"GENERATED DOCSTRING:\n{gen['docstring']}\n\n"
        content += f"{'-'*80}\n\n"

    timestamp = datetime.now().strftime("%Y-%m-%d_%H-%M-%S")
    filename = f"doc_genie_{timestamp}.txt"

    try:
        Path(filename).write_text(content)
        return f"✅ TXT Downloaded: {filename}", filename
    except Exception as e:
        return f"❌ Error: {str(e)}", None

def generate_whatsapp_link():
    """Generate WhatsApp share link"""
    if not generation_history:
        return "❌ No documentation to share!"

    message = "📚 *Doc-Genie Documentation*\n\n"
    for i, gen in enumerate(generation_history[-3:], 1):
        message += f"*Doc {i}:*\n"
        message += f"Style: {gen['style']}\n"
        message += f"`{gen['code'][:60]}...`\n\n"

    encoded = urllib.parse.quote(message)
    link = f"https://wa.me/?text={encoded}"

    return f"✅ WhatsApp Link:\n\n{link}\n\n📱 Share with team!"

def generate_facebook_link():
    """Generate Facebook share link"""
    if not generation_history:
        return "❌ No documentation to share!"

    message = f"I generated {len(generation_history)} professional Python function docstrings with Doc-Genie! Check
    encoded = urllib.parse.quote(message)
    link = f"https://www.facebook.com/sharer/sharer.php?quote={encoded}"

    return f"✅ Facebook Link:\n\n{link}\n\n👍 Share on Facebook!"
```

Code reference (hint: click on image and you can copy paste from Comment)

```python
print("🚀 Loading Doc-Genie Improved Version...")

with gr.Blocks(
    title="Doc-Genie - Python Documentation Generator",
    theme=gr.themes.Soft(),
    css="""
    body { background: linear-gradient(135deg, #1a1a2e 0%, #16213e 100%); }
    .gradio-container { background: linear-gradient(135deg, #1a1a2e 0%, #16213e 100%); }
    .button-primary { background: linear-gradient(135deg, #FF6B6B 0%, #FF8E72 100%); }
    .button-secondary { background: linear-gradient(135deg, #4ECDC4 0%, #44A08D 100%); }
    .code-block { background: #0f3460; border: 2px solid #4ECDC4; border-radius: 8px; padding: 15px; }
    """
) as demo:

    gr.Markdown("""
    # 📚 Doc-Genie v2.0
    ## Professional Python Function Docstring Generator

    Transform your Python functions with AI-powered, accurate documentation!
    """)

    with gr.Row():
        with gr.Column(scale=3):
            # Code Input Area
            gr.Markdown("### 📝 Input Function Code")

            code_input = gr.Code(
                language="python",
                label="Python Function",
                lines=10,
                value="def calculate_factorial(n):\n    if n == 0:\n        return 1\n    return n * calculate_factorial(n-1)"
            )
```

Code reference (hint: click on image and you can copy paste from Comment)

MILESTONE 4: Gradio Interface & User Experience

Interface Components Overview

Main Interface Sections:

1. LEFT COLUMN (Code Input/Output):
   - Code input area with syntax highlighting
   - File upload for .py files
   - Style selector (Google/NumPy radio)
   - Generate button
   - Code output display

2. RIGHT COLUMN (Controls & Export):
   - Clear history button
   - PDF download button
   - TXT download button
   - WhatsApp share button
   - Facebook share button
   - Feature information panel

Activity 4.1: Core Input/Output Interface

Code Input Component:
- Textarea with Python syntax highlighting
- Pre-filled with example function
- Support for multi-line code
- Line numbers and formatting

File Upload:
- Accept .py file format
- Read and display file content
- Error handling for invalid files

Style Selector:
- Radio buttons: "google" and "numpy"
- Real-time style switching
- Default: "google" style

Output Display:
- Syntax-highlighted code block
- Shows function with generated docstring
- Copy-paste ready format

```python
# File Upload
file_upload = gr.File(label="📤 Or Upload .py File", file_types=['.py'])
upload_status = gr.Textbox(label="Upload Status", interactive=False, lines=1)

def process_and_display(file):
    if file:
        content, status = process_upload(file)
        return content, status
    return "", "No file"

file_upload.change(process_and_display, inputs=[file_upload], outputs=[code_input, upload_status])

# Style Selection
with gr.Row():
    docstring_style = gr.Radio(["google", "numpy"], value="google", label="🎨 Docstring Style")
    generate_btn = gr.Button("✨ Generate Docstring", size="lg", variant="primary", scale=2)

# Output Area
gr.Markdown("### 📄 Generated Output")

code_output = gr.Code(
    language="python",
    label="Function with Docstring",
    lines=15,
    interactive=False
)
```

Code reference (hint: click on image and you can copy paste from Comment)

```python
        lines=15,
        interactive=False
    )

    gen_status = gr.Textbox(label="Status", interactive=False, lines=1)

    generate_btn.click(
        generate_docstring,
        inputs=[code_input, docstring_style],
        outputs=[code_output, gen_status]
    )

with gr.Column(scale=1):
    gr.Markdown("### 🎮 Controls")

    clear_btn = gr.Button("🗑️ Clear History", variant="stop", size="sm")
    clear_btn.click(clear_history, outputs=[upload_status])

    gr.Markdown("---")
    gr.Markdown("### 📥 Export")

    pdf_btn = gr.Button("📄 Download PDF", size="sm", variant="primary")
    txt_btn = gr.Button("📋 Download TXT", size="sm", variant="primary")

    pdf_file = gr.File(label="PDF File", visible=True)
    txt_file = gr.File(label="TXT File", visible=True)
```

```
txt_btn.click(download_txt, outputs=[export_status, txt_file])

gr.Markdown("---")
gr.Markdown("### 📤 Share")

whatsapp_btn = gr.Button("💬 WhatsApp", size="sm", variant="secondary")
facebook_btn = gr.Button("👍 Facebook", size="sm", variant="secondary")

share_output = gr.Textbox(label="Share Link", interactive=False, lines=4)

whatsapp_btn.click(generate_whatsapp_link, outputs=[share_output])
facebook_btn.click(generate_facebook_link, outputs=[share_output])

gr.Markdown("---")
gr.Markdown("""
### 📊 Features

**Improvements:**
- ✅ Accurate logic analysis
- ✅ Contextual descriptions
- ✅ Operation detection
- ✅ Loop & condition recognition
- ✅ Better PDF export
- ✅ Consistent output
```

Code reference (hint: click on image and you can copy paste from Comment)

```
### 🎯 Supported Styles

**Google:** Standard format
**NumPy:** Scientific format
""")

print("\n✅ Doc-Genie v2.0 Ready!\n")
demo.launch(share=True, show_error=True, show_api=False)
```

Code reference (hint: click on image and you can copy paste from Comment)

Activity 4.2: Export & Sharing Features

PDF Export:
- ReportLab-based document generation
- Professional styling and layout
- Timestamp and metadata inclusion
- Code block formatting
- Pagination for multiple docs

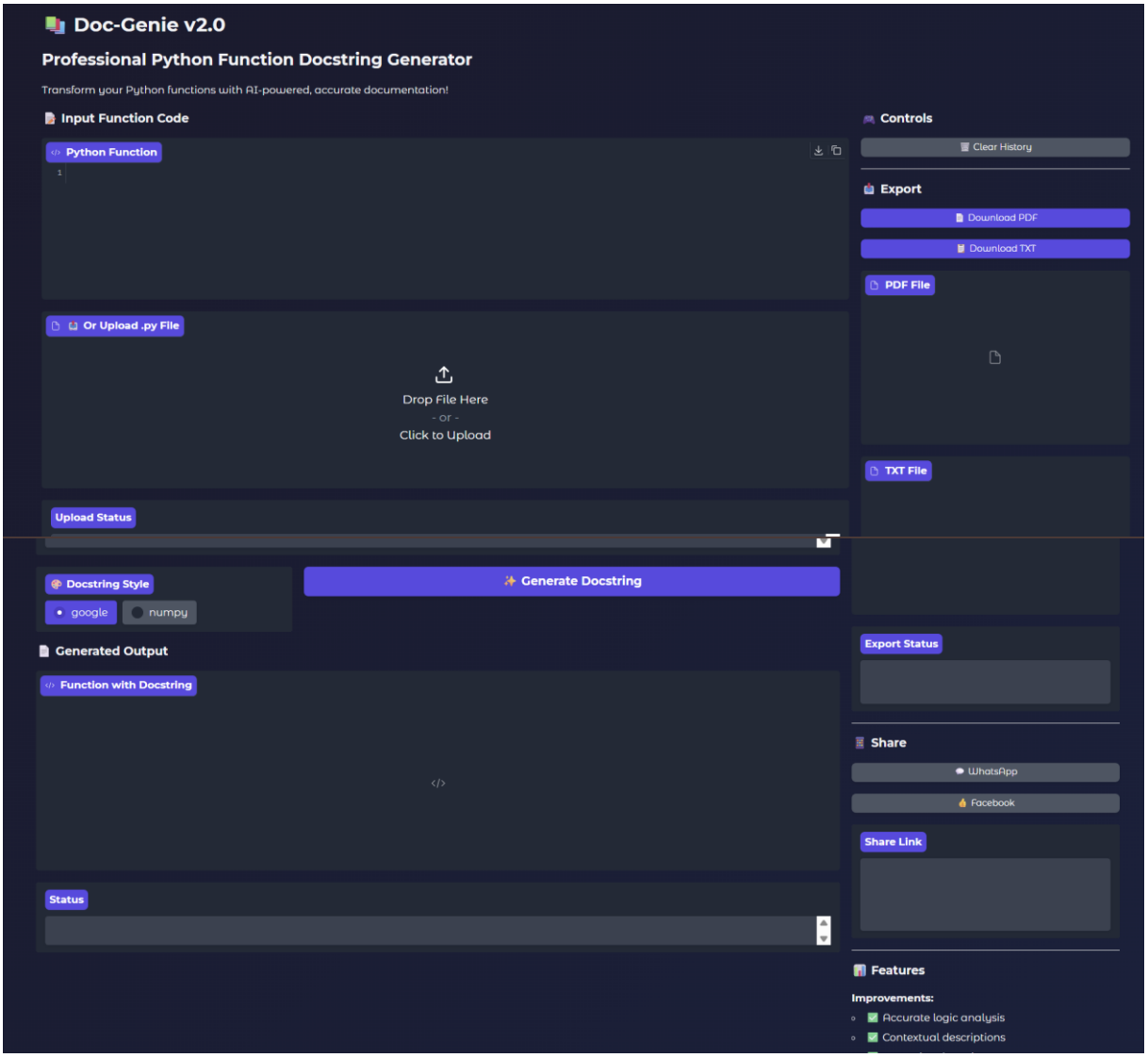
TXT Export:
- Plain text format with clear separators
- Includes original code and docstring
- Preserves formatting for readability
- Batch export support

Social Sharing:
- WhatsApp link with message preview
- Facebook share with summary
- URL encoding for special characters
- Message customization

Code reference (hint: click on image and you can copy paste from Comment)

MILESTONE 5: Testing & Deployment

Activity 5.1: Google Colab Deployment



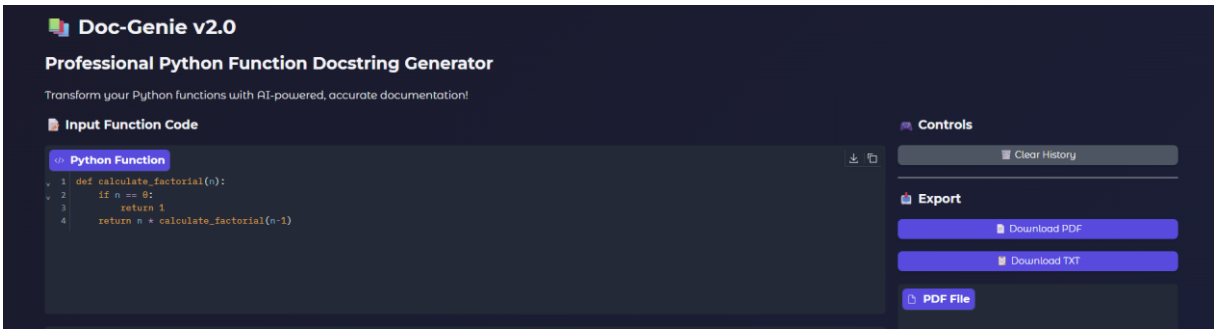
public URL: https://3ab3868bb13f172119.gradio.live

Activity 5.2: Functional Testing

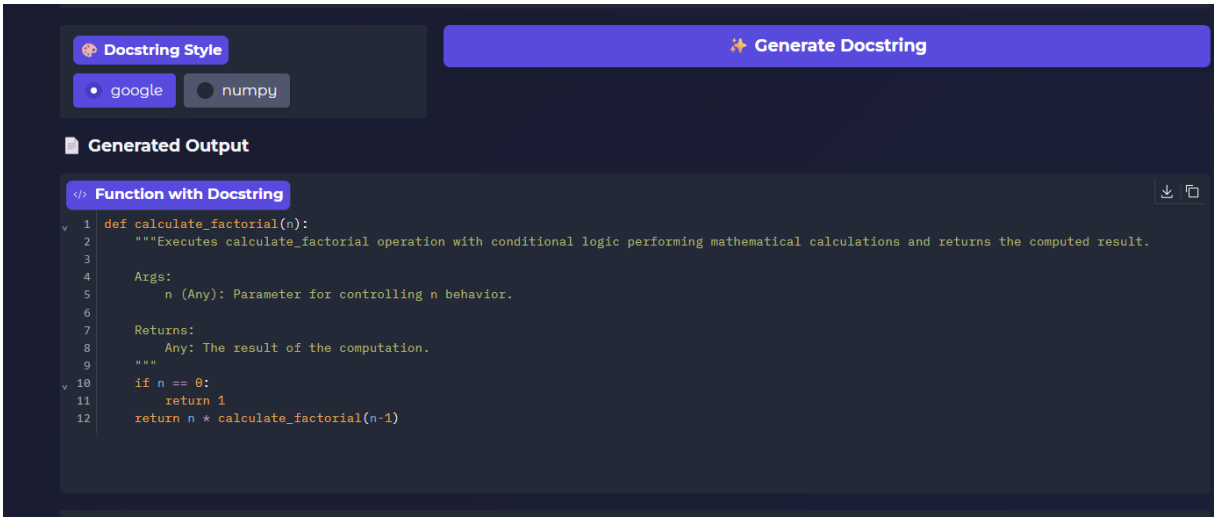
Test Case 1: Simple Function Documentation

Input Code:

```
def calculate_factorial(n):

    if n == 0:

        return 1

    return n * calculate_factorial(n-1)
```



Generated Google Docstring(output):



Status: ✅Correct docstring generation

FUTURE ENHANCEMENTS

The modular architecture allows for extension with features such as support for additional programming languages (Java, C++, JavaScript), integration with IDE plugins (VSCode, PyCharm), advanced machine learning-based function description generation, support for class and module documentation, automatic documentation deployment to GitHub/GitLab, integration with documentation generators (Sphinx, MkDocs), and real-time code quality analysis with documentation metrics.

With continued development, Doc-Genie can evolve into a comprehensive code documentation platform supporting diverse programming paradigms while maintaining its focus on accuracy, ease of use, and professional documentation standards. The intelligent analysis engine provides a foundation for future features including AI-powered best practice recommendations, documentation style enforcement, and automated technical debt identification.

CONCLUSION

Doc-Genie is an intelligent tool that helps developers automatically generate documentation for Python functions. It uses AST analysis to understand Python code and create accurate docstrings.

The system provides a simple interface using Gradio, making it easy for developers, students, and teams to generate documentation quickly. It supports multiple documentation styles and allows users to export the documentation as files.

Overall, Doc-Genie makes the process of writing Python documentation faster, easier, and more organized, helping developers maintain well-documented and professional code.